IT305 — Computer Networks Laboratory

Fault-Tolerant Topic-Based File Distribution Service

Final Technical Report

Group 050 & 055

Full name	Roll number
Bhagya Sheth	202401200
Om Shiloja	202401201
Dhruv Sodvadiya	202401207
Parth Sorathiya	202401215
Prins Satapara	202401188
Dhaval Popat	202401161

Prepared from the supplied Part 1 source archive, Part 2 source archive, and results archive.

23 October 2026

Contents

1 Executive Summary **2**

2 Objectives and Scope **2**

3 System Design and Experimental Environment **2**

3.1 Architecture . . . 2
3.2 Dataset and setup . . . 2
3.3 Project organization . . . 3
3.4 Implementation-level protocol and data handling . . . 3
3.5 Concurrency and operating system interactions . . . 3
3.6 Deviation register: earlier design versus delivered implementation . . . 3
3.7 Metrics and overhead definitions . . . 4

4 Part 1 — Topic-Based File Distribution **4**

4.1 Implementation . . . 4
4.2 Performance methodology . . . 4
4.3 Part I overhead and queueing effects . . . 6
4.4 Interpretation . . . 6

5 Part 2 — Fault Injection and Recovery **6**

5.1 Case 1: Whole-topic restart . . . 6
5.1.1 Additional Case 1 recorded runs . . . 7
5.2 Case 2: Checkpoint resume . . . 8
5.2.1 Staged checkpoint-resume evidence . . . 8
5.2.2 Case 2 algorithm and overhead interpretation . . . 8
5.3 Enhanced case: Parallel transfer and recovery . . . 10
5.3.1 Enhanced implementation and overhead interpretation . . . 10
5.3.2 Measured counters and blocking-I/O classification . . . 12
5.4 Summary of observed Part II results . . . 12

6 Cross-Case Comparison and Limitations **12**

6.1 Part II evidence and experimental comparability . . . 13

7 Validation and Reproducibility **14**

8 Individual Contributions **15**

9 Results Coverage Against Assignment Criteria **15**

10 Conclusion **16**

1 Executive Summary

This project implements a topic-based TCP file distribution service in C11. A client selects an animal topic (e.g., `cane` or `gatto`) and retrieves the files in that directory from a server. Part 1 investigates sequential and concurrent client handling; Part 2 introduces probabilistic transfer failures, whole-topic restart, checkpoint-based resume, and parallel-connection recovery. The submitted materials contain source code, CSV results, and generated performance figures.

The team established a three-laptop test environment for the Part 1 workflow (one server and two client hosts). The results archive also includes Case 1, Case 2, and Enhanced measurements; individual dataset and connectivity contexts differ across those observations. Accordingly, this report differentiates *measured evidence*, *implementation behavior*, and *limitations* rather than treating unlike runs as a controlled head-to-head benchmark.

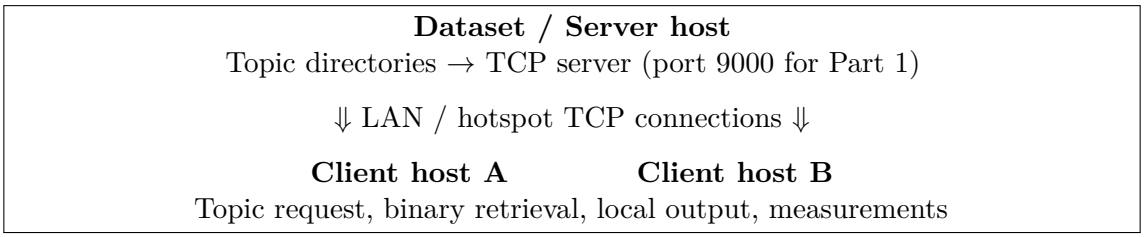
2 Objectives and Scope

- Support topic-based file requests and complete binary file transfers over TCP.
- Contrast single-threaded and multi-threaded handling as client concurrency changes.
- Inject failures during transfer and assess retry behavior and data-transfer overhead.
- Resume downloads from persistent client-side checkpoints without requiring the server to retain connection-specific session state.
- Enable parallel transfer using four connections per client in the Enhanced case.
- Collect reproducible measurements, check transferred file integrity, and present performance graphs.

3 System Design and Experimental Environment

3.1 Architecture

The server enumerates topic directories in the supplied dataset root. Clients issue a textual topic request and receive file metadata followed by file bytes over TCP. Common networking and topic-scanning utilities are shared within each code package. The source archives include Linux-compatible Makefiles, C11 sources and local test scripts.



3.2 Dataset and setup

Animals-10 supplies topic folders with JPEG images; recorded Part 1/Part 2 experiments include `cane` with 4,863 files and 60,686,716 unique payload bytes. The testing narrative documents Windows laptops running Ubuntu through WSL2, LAN/hotspot networking, Windows firewall rules and port forwarding to WSL2. A successful local transfer and remote client connections were separately observed. Experiment-specific environment and network conditions should be retained alongside raw data for strict reproducibility.

3.3 Project organization

The provided Part 1 code archive contains `Part1/`, `common/` and `tests/`. The Part 2 archive contains `Part2_Case1/`, `Part2_Case2/`, `Part2_Case2_Enhanced/`, shared utilities and tests. The third archive contains CSVs and plots for all variants. These were delivered as separate packages and should be integrated and revalidated as a final build before submission.

3.4 Implementation-level protocol and data handling

The earlier design specifies a directory-to-topic map, a deterministic sorted manifest, newline-delimited control headers, and unmodified binary file payloads. The source implements topic enumeration using directory traversal and filename validation, followed by a sorted file manifest. A client sends `GET <topic>` and receives `OK <count>`, then for each file `FILE <name> <size>` followed by exactly `size` bytes; `DONE` terminates the stream. Invalid requests or absent topics generate `ERR` responses. Because each file length is transmitted explicitly, the receiver distinguishes raw image bytes from later control lines without relying on JPEG markers or TCP packet boundaries.

The server uses a bounded 64-KiB transfer buffer and repeat-until-complete socket sends. The client reads the exact promised length and writes the bytes to a topic-specific output directory. The topic list and per-request file list are sorted for deterministic ordering, which also makes file-based restart and checkpoint lookup predictable. File metadata is enumerated per topic request; the design's conceptual topic map is realized as a startup-scanned directory inventory rather than a network-visible database.

3.5 Concurrency and operating system interactions

The sequential mode accepts a TCP connection, runs its entire client handler and closes the socket before accepting another service request. The concurrent mode dispatches accepted sockets to POSIX worker threads; shared topic metadata is read-only once initialized, whereas a request's file list, socket, transfer buffer and counters are connection-local. The executable mode selection in the supplied Part I package is an optional command-line mode, and the independently compiled `server_single` and `server_multi` binaries used during development provide the same experimental distinction. Thread-per-connection keeps implementation straightforward but is not a bounded thread-pool design.

3.6 Deviation register: earlier design versus delivered implementation

Earlier design	**Delivered implementation / observation**	**Consequence**
Case 2 proposes logical session management and a client checkpoint	Server reconstructs progress from each `RESUME` message; client stores durable progress; no server-side persistent session registry	Session continuity is logical and client-driven rather than server-stateful.
Enhanced design describes no separate control connection	The actual client opens a short-lived `META <topic>` connection for file count and manifest version, closes it, and then launches `PARALLEL` data workers	A metadata control handshake exists, though not as a persistent coordinator connection.
The design describes a simple resume tuple including a filename and offset	Source adds a manifest token to resumed requests, providing a validation input to the server	More explicit stale-manifest detection than the simplified design message table.

Earlier design	Delivered implementation / observation	Consequence
The design's checkpoint example names a topic-specific binary checkpoint file	Source uses the `checkpoint.c` helper and hidden checkpoint files under the output/checkpoint directory, with per-connection storage for Enhanced	Artifact location and serialization should be understood from the source rather than assumed from the design illustration.
The design proposes comparative throughput and transfer overhead	Archived CSVs measure Case 1 redundant bytes; complete cross-reconnect transmitted-byte totals for Case 2/Enhanced are not available	Quantitative redundancy is unavailable for two variants, a measurement limitation rather than a zero-overhead finding.
Design specifies an optional server chunk-size argument	Actual Part II server CLI accepts port, topic-root and failure probability only; chunking is configured in the source	The design CLI and implementation interface differ.
Design proposes an optional Case 2 checkpoint-byte setting	Case 2 client exposes an optional retry bound, not a checkpoint-byte CLI parameter	Checkpoint granularity is not user-adjustable through the described CLI.
Enhanced design discusses send and receive socket timeouts	Client uses blocking sockets with a 10-second receive timeout; explicit send timeout was not found	Document actual socket behavior and avoid claiming non-blocking I/O.

3.7 Metrics and overhead definitions

Across all variants, completed payload bytes are counted separately from retransmitted bytes where the counters permit. For a completed topic of U distinct original bytes and observed total transmitted bytes T, retransmission overhead is $R = T - U$ and overhead ratio is $100R/U$ percent. For batches, aggregate throughput $= \sum U/(10^6 \Delta t)$, using the elapsed time from the first batch start to the last client's completion. *Protocol overhead* (TCP/IP headers, ACKs, retransmissions below the application layer) would require a packet capture and is not quantified by these application counters. Under fault-free Part I, no application-triggered restart overhead is expected, but its packet-level overhead was not recorded. This distinction prevents falsely equating elapsed-time differences with measured wasted bytes.

4 Part 1 — Topic-Based File Distribution

4.1 Implementation

The supplied Part 1 source package supports the required `server <port> <topics_root_dir>` and `client <server_ip> <port> <topic> <output_dir>` interfaces. Its server also accepts an optional `single` or `multi` mode. In the single-threaded design, clients are processed sequentially after `accept()`. In multi-threaded mode, connection work can overlap using POSIX threads. The client's terminal metrics report completed files, transferred bytes, elapsed time and throughput.

4.2 Performance methodology

The comparison CSV contains batches with 1, 2, 4, 6 and 8 clients per server mode. Aggregate throughput is computed from the total transferred payload and elapsed batch wall time:

$$\text{Throughput}_{\text{aggregate}} = \frac{\text{successful payload bytes}}{10^6 \times \text{batch duration (s)}}.$$

This is distinct from averaging individual client rates or the server's per-connection transfer duration. The observations below are transcribed directly from `Part1_Comparison.csv`; they do not establish

identical network conditions between runs.

Mode	Clients	Total bytes	Batch time (s)	MB/s
Single-threaded	1	60,686,716	32.435	1.871
Single-threaded	2	98,107,225	54.925	1.786
Single-threaded	4	218,494,065	229.011	0.954
Single-threaded	6	421,480,771	257.836	1.635
Single-threaded	8	509,943,645	304.707	1.674
Multi-threaded	1	60,686,716	166.383	0.365
Multi-threaded	2	98,107,225	82.005	1.196
Multi-threaded	4	218,494,065	113.688	1.922
Multi-threaded	6	421,480,771	494.637	0.852
Multi-threaded	8	509,943,645	63.484	8.033

Table 2: Recorded Part 1 aggregate comparison results.

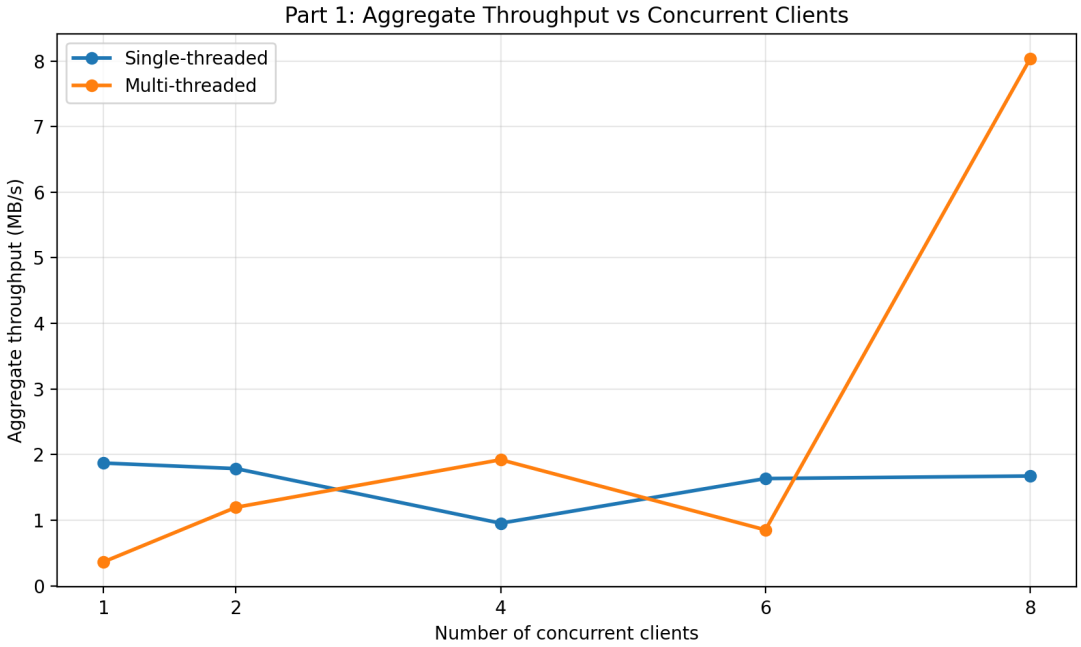


Figure 1: Part 1 aggregate throughput comparison, as provided in the results archive.

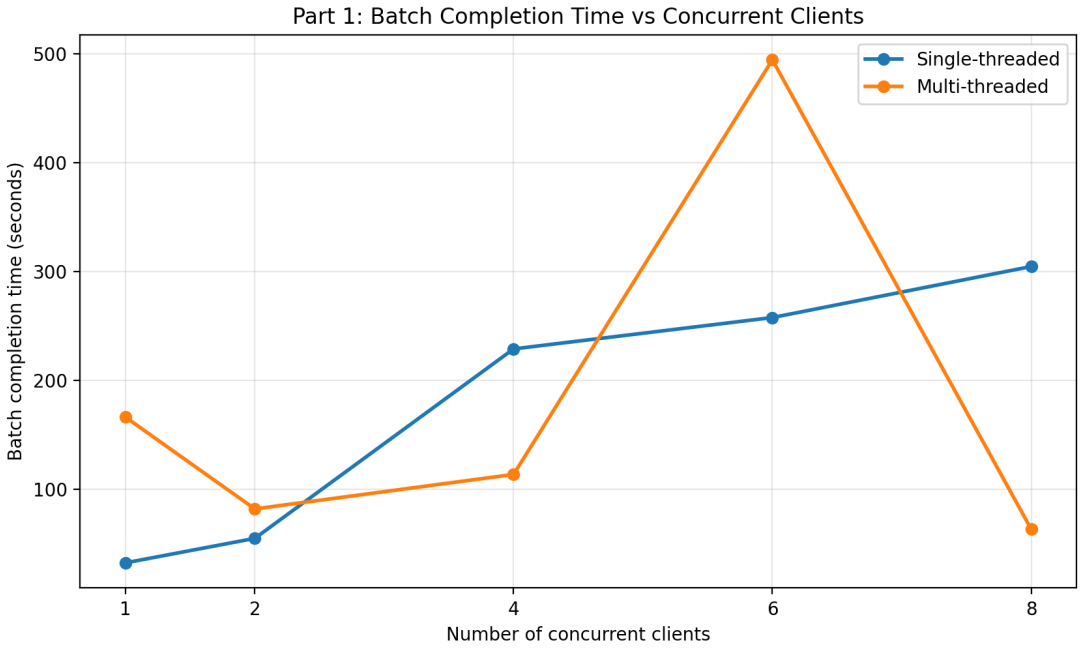


Figure 2: Part 1 batch completion time versus client count.

4.3 Part I overhead and queueing effects

The performance archive reports payload bytes, batch completion time and normalized aggregate throughput, but does not include wire-level header/ACK byte counts or an application retransmission counter for Part I. Thus the overhead evidence is presented as *elapsed batch time and contention/queueing delay*, not as a fabricated retransmitted-byte percentage. The transfer-time graph above is the measured timing-overhead comparison. In sequential mode, later accepted work cannot begin file delivery until the active handler completes; in concurrent mode, workers can overlap but compete for Wi-Fi and disk resources.

4.4 Interpretation

At eight clients the recorded multi-threaded batch attained 8.033 MB/s versus 1.674 MB/s for the single-threaded batch. At six clients the opposite trend occurred (0.852 versus 1.635 MB/s). A large difference is also visible at one client. Therefore the results show variability and cannot support a universal claim that threading is always faster. Wi-Fi contention, workload composition, simultaneous start alignment, machine load and measurement procedure are plausible confounders. The next reproducibility step is to repeat matched scenarios under stable network conditions.

5 Part 2 — Fault Injection and Recovery

5.1 Case 1: Whole-topic restart

The Case 1 server checks a configured fault probability after transmitted chunks of up to 64 KiB. Following an injected disconnection, the client retries the topic from its beginning; partial work from failed attempts does not become a resumable checkpoint. The implementation supports bounded retries and reports received, useful and redundant bytes.

The archived `cane` runs at probability 0.0002 show client A completing after one retry (16.160 s, 41,341,614 redundant bytes), and client B after seven retries (74.318 s, 148,466,451 redundant bytes).

These values demonstrate the cost of restarting transferred content. Another archive entry records a probability-0.001 trial that exhausted retries and did not complete; unsuccessful trials should not be omitted when discussing reliability.

Client	Fault p	Retries	Duration (s)	Normalized MB/s	Redundant MB
A	0.0002	1	16.160	3.755	41.342
B	0.0002	7	74.318	0.817	148.466

Table 3: Case 1 measured restart overhead and unique-payload goodput; source: archived cross-case CSV.

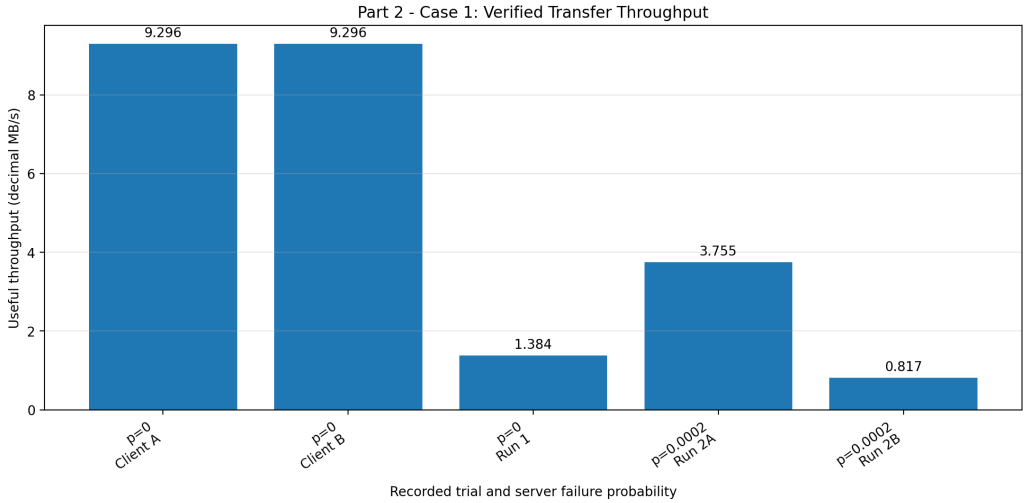


Figure 3: Case 1 throughput results from the supplied archive.

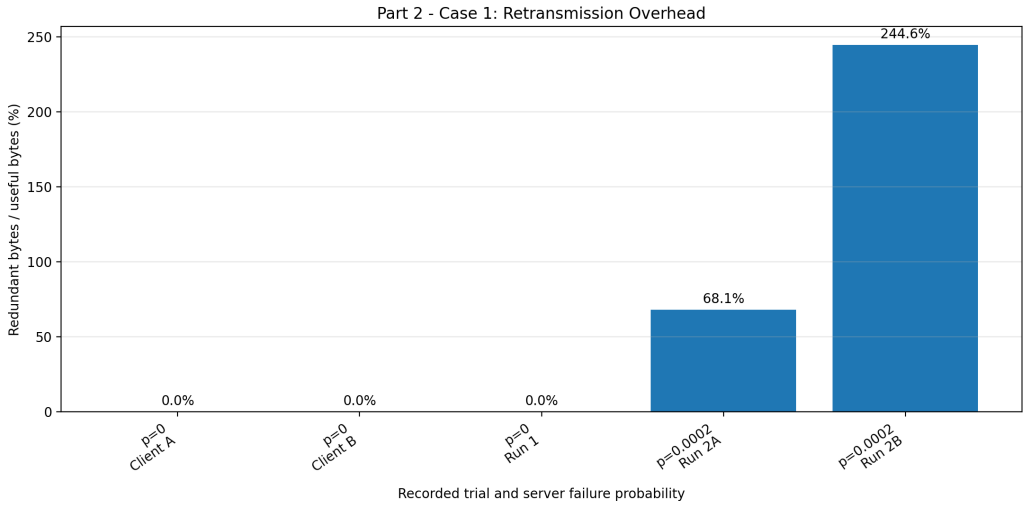


Figure 4: Case 1 reported overhead graph from the supplied archive.

5.1.1 Additional Case 1 recorded runs

The Part II reference also reports a single-client no-fault run (43.855 s, 1.384 MB/s), a two-client no-fault trial (6.529 s / 9.296 MB/s for Client A and 6.528 s / 9.296 MB/s for Client B), and a retry-limit failure at $p = 0.001$ (10 retries, 289.364 s, 126,706,802 reported redundant bytes and no completed topic). These conditions are not interchangeable with the faulted $p = 0.0002$ run: network load and fault regimes differ. The error run is retained rather than excluded from the reliability findings.

5.2 Case 2: Checkpoint resume

Case 2 adds a persisted client-side checkpoint, a `RESUME` request and server-side validation of resume information without retaining an active session between connections. The supplied Case 2 records include forced faults at probability 1.0, which produced a reported checkpoint offset of 9,451 bytes, followed by successful resume-only phases under probability 0.0. That two-stage demonstration verifies recovery behavior but is *not* a same-probability throughput trial and excludes initial downtime from resume-only elapsed time.

The archive also records two successful automatic-recovery sessions at probability 0.001, with five and six client-reported retries and elapsed times 115.219 s and 124.425 s. Their raw client file/useful counters exceed the source dataset's unique-file/unique-byte totals; a corrected, normalized throughput measure is therefore used in the comparison rather than treating these counters as independent unique payload.

5.2.1 Staged checkpoint-resume evidence

In a separate protocol-validation test, both clients were first interrupted with $p = 1.0$ and saved a checkpoint at byte offset **9,451**. The server was then restarted with $p = 0$, and both logs showed an accepted `RESUME` at that non-zero offset. Recorded resume-phase durations were 92.909 s and 92.934 s (normalized rate approximately 0.653 MB/s for each). These durations exclude the initial fault phase and server-restart downtime, and therefore are not used as end-to-end performance measurements against the automatic-recovery trials.

The automatic $p = 0.001$ run reported 4,868 files / 60,745,977 bytes for Client A and 4,869 files / 60,767,371 bytes for Client B. Both exceed the distinct source inventory. Source-byte normalization corrects the throughput denominator without asserting that the raw client completion counters measure unique files.

5.2.2 Case 2 algorithm and overhead interpretation

The client loads a checkpoint containing the current topic, the last file and committed offset. If no valid checkpoint exists it starts with `GET`; otherwise it sends `RESUME` and supplies the saved position and manifest token. The server recomputes the ordered topic file list, validates the position and sends data beginning at the accepted offset. The client persists progress on successful chunk boundaries and after file completion, and removes the checkpoint when `DONE` is reached. On stale state the client falls back to a fresh range/topic transfer according to the code path.

Case 2's automatic-recovery observations at $p = 0.001$ are summarized below. Source-file normalized goodput uses 60,686,716 bytes as the unique useful payload, rather than the inflated post-resume client counters.

Client	Retries	Elapsed (s)	Unique-payload MB/s	Redundant bytes
A	5	115.219	0.527	Not recoverable
B	6	124.425	0.488	Not recoverable

Table 4: Case 2 auto-recovery: actual recorded durations and normalized goodput; redundancy is not reliably measurable from retained logs.

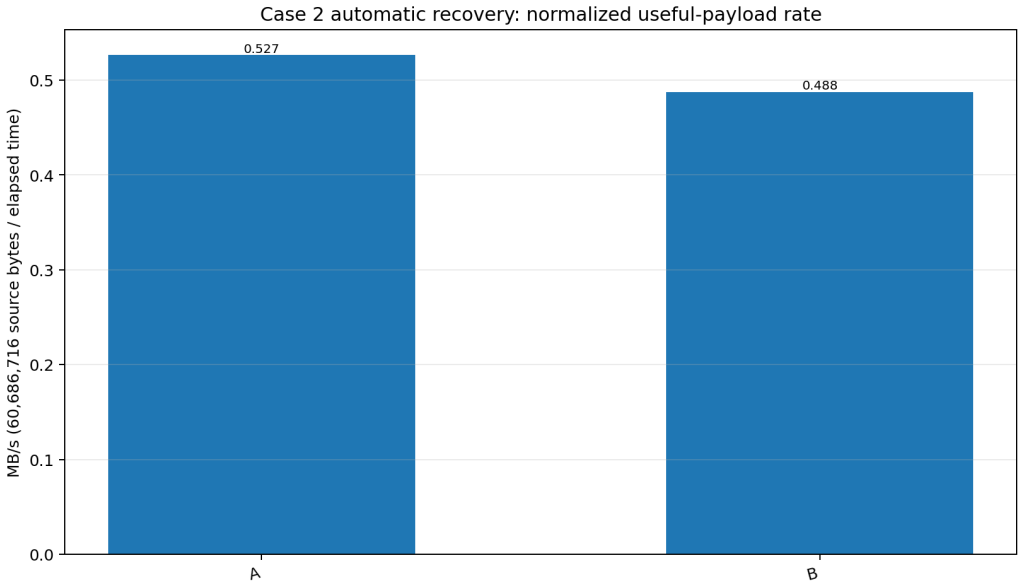


Figure 5: Case 2 unique-payload-normalized throughput.

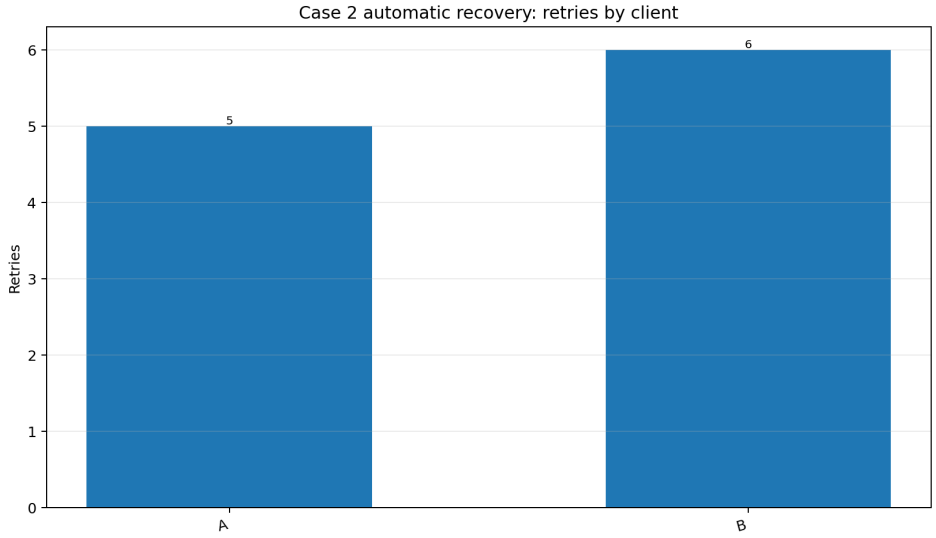


Figure 6: Case 2 retry count as the available recovery-effort proxy; not a byte-overhead measurement.

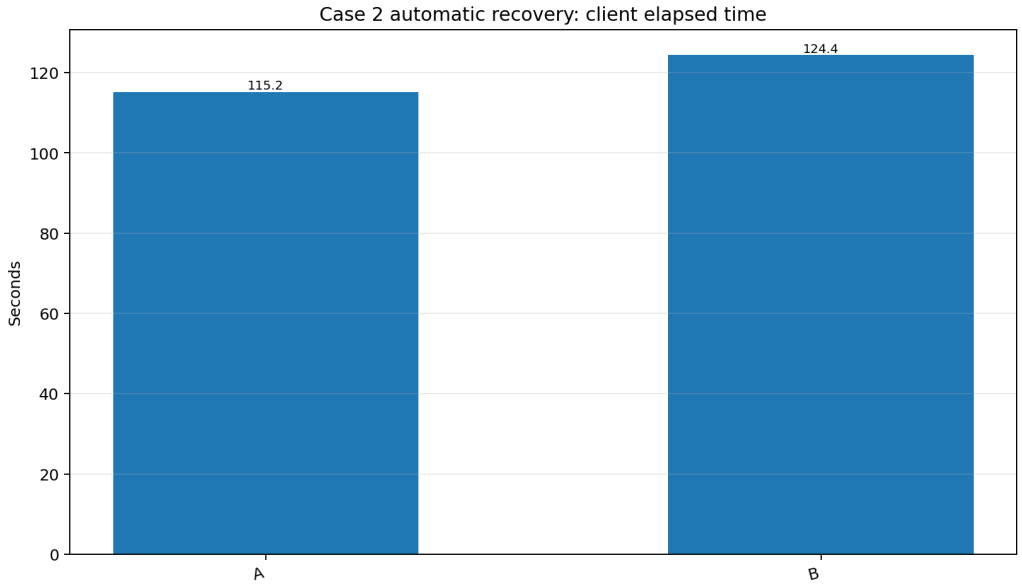


Figure 7: Case 2 automatic-recovery timings supplied with the experiment results.

5.3 Enhanced case: Parallel transfer and recovery

The Enhanced implementation includes a short-lived `META` request for metadata, followed by parallel TCP workers and `RESUME PARALLEL` requests with range/offset information. The archived experimental scenario uses four connections per client and two concurrent client hosts. At probability 0.001, the client-observed completion times were 25.680 s and 25.629 s, with four and three retries, respectively. The corrected normalized unique-payload rates are 2.363 and 2.368 MB/s. The archived status note for this directory says "planned", but the actual included source contains parallel worker and resume code; that note is stale and should be corrected in the submission package.

5.3.1 Enhanced implementation and overhead interpretation

After closing the `META` connection, the client creates four parallel workers by default; each opens its own TCP connection, identifies its connection index and total worker count, and receives a deterministic contiguous range of complete files. Each worker maintains an independent checkpoint. After a fault, only that worker's assigned range must be reconnected and resumed; other transfers may continue. Whole-file partitioning avoids a file-fragment reassembly stage and maintains distinct output filenames. Connection count is a controllable argument, not necessarily four in every possible invocation.

Client	Connections	Retries	Seconds	Goodput MB/s	Byte overhead
A	4	4	25.680	2.363	Not measured
B	4	3	25.629	2.368	Not measured

Table 5: Enhanced case: four-connection observations at fault probability 0.001.

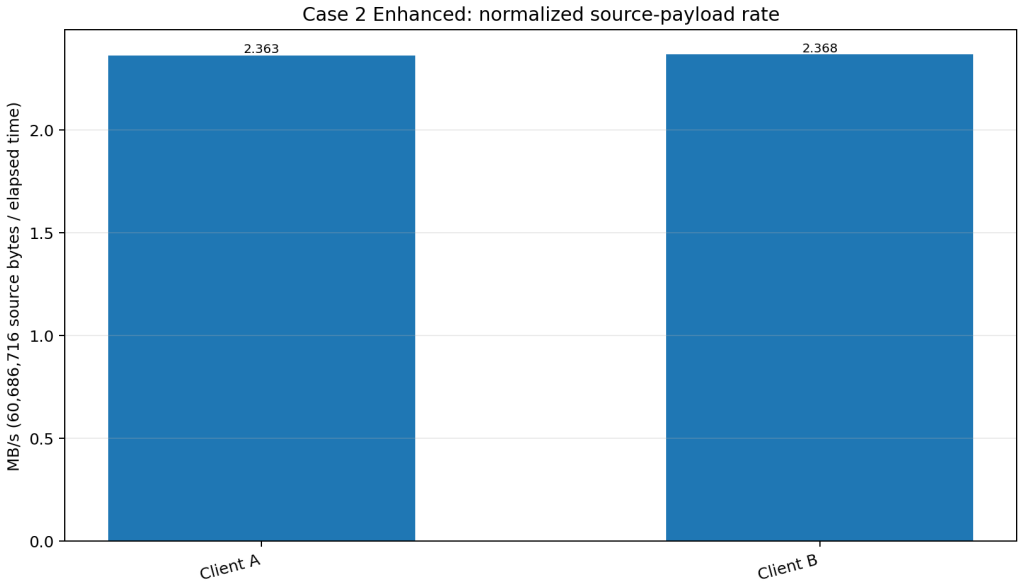


Figure 8: Enhanced case unique-payload-normalized throughput.

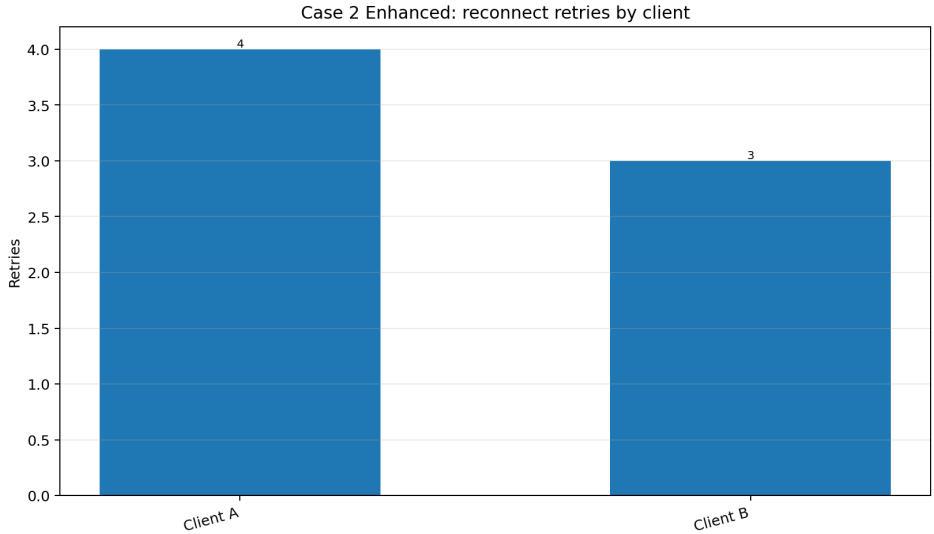


Figure 9: Enhanced case recovery activity (retries), the available proxy when redundant byte totals are missing.

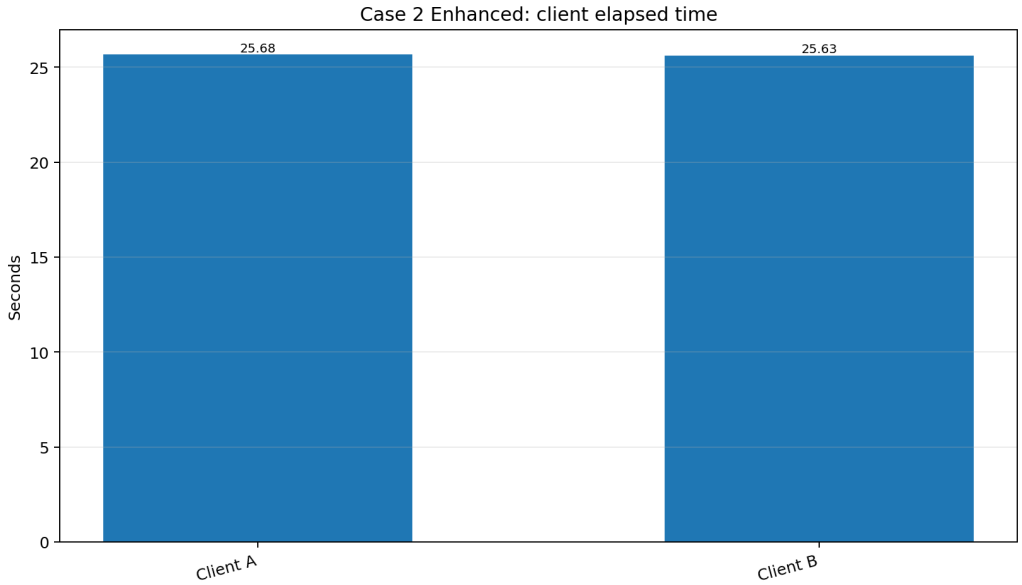


Figure 10: Enhanced case observed transfer durations.

5.3.2 Measured counters and blocking-I/O classification

The four-connection trial reported **4,867 files / 60,738,750 bytes** for Client A and **4,866 files / 60,715,664 bytes** for Client B, compared with **4,863 distinct files / 60,686,716 unique source bytes**. These inflated completion counters most likely re-count resume-anchor files, so the normalized rate uses the distinct source-payload denominator rather than those raw values. They must not be described as independent additional source files. Final process status was OK for both clients, but the archived screenshot did not retain SHA-256 verification output.

The Enhanced implementation uses **blocking TCP sockets**, with `SO_RCVTIMEO` set to ten seconds on the client. The inspected source does not set `O_NONBLOCK`, nor does it use `poll`, `select`, or `epoll`. An explicit `SO_SNDTIMEO` configuration was not found. These are implementation details and, for send timeout, a documented gap relative to the design text.

5.4 Summary of observed Part II results

Case 1 exposes a measured redundant-byte cost when the client restarts whole-topic downloads. Case 2 recovers from persisted offsets and has automatic recovery evidence, but no reliable accumulated network-byte counter across repeated connections. Enhanced distributes whole-file ranges among workers and records faster completed transfers than Case 2 in the supplied scenarios, while exact retransmission overhead remains unknown. Consequently this is evidence of working recovery strategies, not a controlled proof that parallelism or checkpoints dominate in every environment.

6 Cross-Case Comparison and Limitations

Case	Recovery mechanism	Client A (s)	Client B (s)
Case 1	Restart entire topic	16.160	74.318
Case 2	Resume from checkpoint	115.219	124.425
Enhanced	Four parallel connections/client with resume	25.680	25.629

Table 6: Selected observed completion times from the archived scenario comparison.

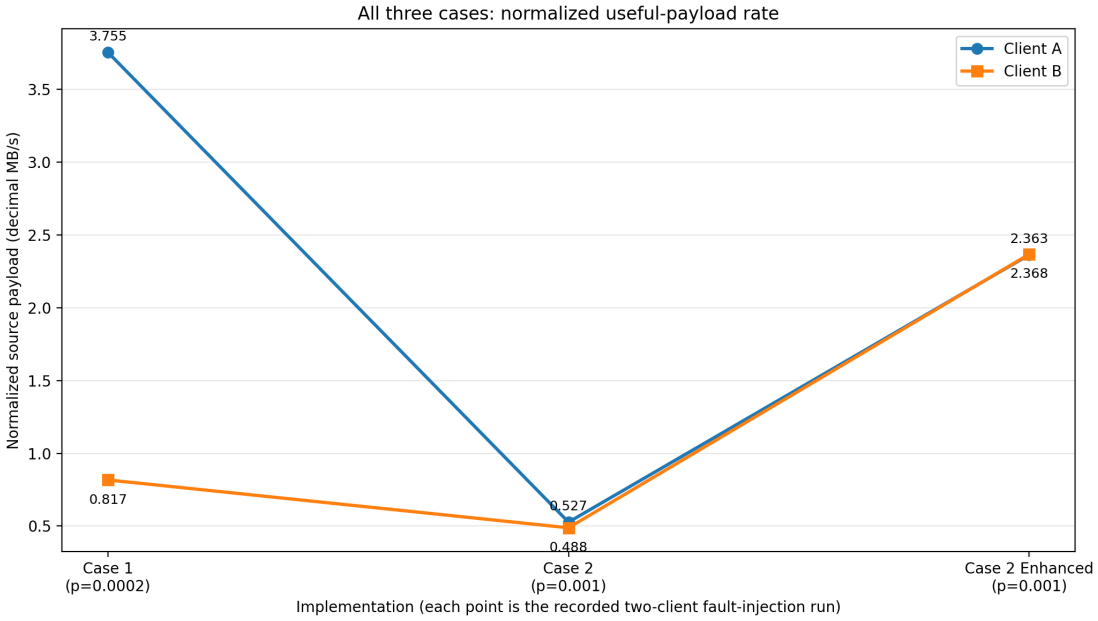


Figure 11: Archived cross-case normalized payload-throughput comparison.

Critical qualification: Case 1 used failure probability 0.0002 while Case 2 and Enhanced used 0.001 in the selected comparison. The runs also differ in transfer parallelism and captured metrics, so these are *descriptive observed scenarios*, not a controlled causal comparison of fault-recovery strategies. Full retransmission overhead cannot be reconstructed reliably for Case 2 and Enhanced from the available cumulative counters. For both variants, client-reported file or useful-byte counts can exceed the source unique totals after resume, and integrity evidence is not uniformly captured within the final archived artifacts.

6.1 Part II evidence and experimental comparability

The Part II reference documents successful C11/pthread builds for all three cases and 14 automated checks in total (five Case 1, four Case 2, five Enhanced). These checks supplement but cannot replace the real-device transfer observations. The recorded Case 1 fault scenario uses $p = 0.0002$, whereas Case 2 and Enhanced use $p = 0.001$; only one principal two-client fault-recovery run is available per configuration. An equal-probability repeated-trial study would be necessary to establish a robust speedup claim.

Case	Client	Failure probability	Retries	Normalized MB/s
Case 1	A	0.0002	1	3.755
Case 1	B	0.0002	7	0.817
Case 2	A	0.001	5	0.527
Case 2	B	0.001	6	0.488
Enhanced	A	0.001	4	2.363
Enhanced	B	0.001	3	2.368

Table 7: Recorded fault-recovery trials, preserving the unequal fault probabilities.

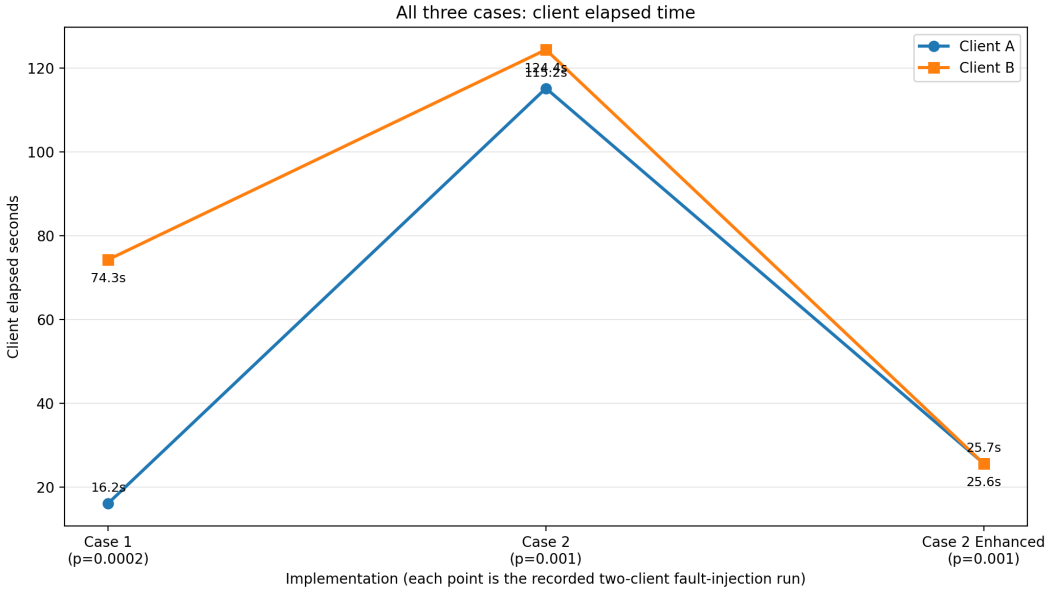


Figure 12: Elapsed-time comparison supplied with the experiments; not a matched fault-probability benchmark.

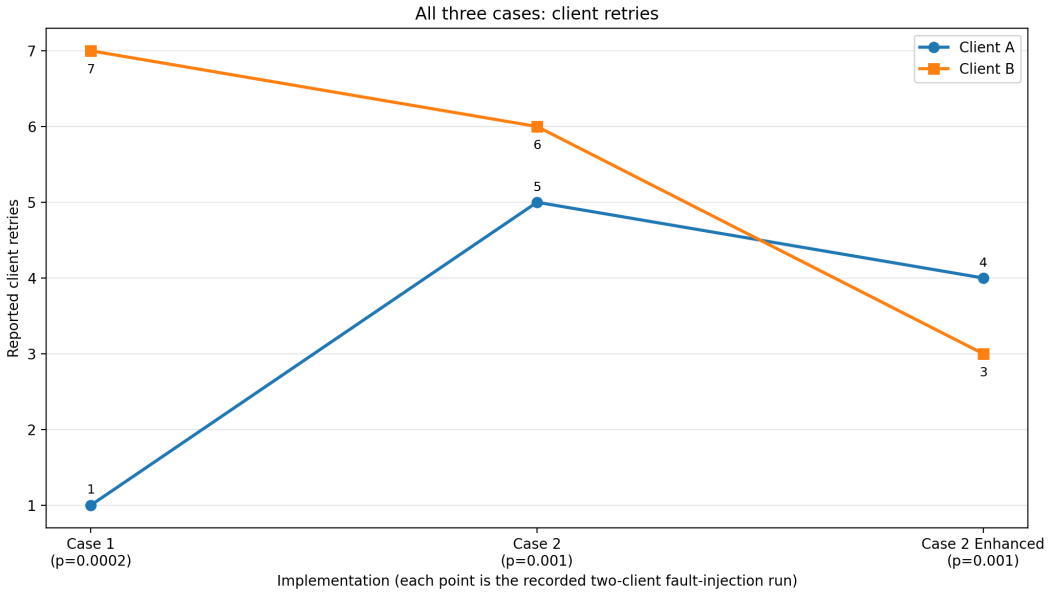


Figure 13: Per-client retry comparison (recovery-effort proxy, not byte-overhead).

7 Validation and Reproducibility

The two source handoffs include automated local test utilities and a file-integrity comparison helper. Part 1 transfer success was observed between physically separate laptop clients. For final evidence, each benchmark should identify the exact source-code revision, topic, payload size, network, failure probability, number of clients, repetition index, start/end measurement scope and verification outcome.

- Rebuild every server and client executable from the exact final archive; confirm that the Part 1 and Part 2 common utilities and protocol assumptions are compatible.
- Repeat each client-count benchmark under matched workloads, logging both individual client records and total batch wall time.

- Verify downloaded files against the corresponding source topic by cryptographic hashes or the bundled verification script.
- Retain raw console logs for injected faults and resume offsets, especially the Enhanced parallel case.
- Keep the supplied CSVs and graph images together in `Results/`; exclude unlabeled synthetic rows from experimental claims.

8 Individual Contributions

Full name and student ID	**Part 1 contribution**	**Part 2 contribution**
Bhagya Sheth 202401200	Source code generation, server–client connectivity establishment, network configuration, debugging, and testing support.	Source code generation for all Part 2 cases, system integration, debugging, and technical support.
Om Shiloja 202401201	Primary involvement in Part 1 execution, testing, performance measurement, results collection, and debugging.	Assisted with testing, debugging, and integration wherever required.
Dhruv Sodvadiya 202401207	Assisted with Part 1 testing and validation.	Major contribution to Part 2 execution, testing, debugging, fault-tolerance verification, and result validation.
Parth Sorathiya 202401215	Assisted with Part 1 testing and validation.	Major contribution to Part 2 execution, testing, debugging, Enhanced case evaluation, and performance analysis.
Prins Satapara 202401188	Assisted with Part 1 testing and validation.	Contributed to Part 2 testing, technical assistance, debugging, and integration support.
Dhaval Popat 202401161	Assisted with Part 1 testing and validation.	Contributed to Part 2 execution, debugging, testing, and verification.

9 Results Coverage Against Assignment Criteria

Case	Throughput evidence	Overhead evidence
Part I	Aggregate throughput table and graph vs client count	Batch-time graph and queueing interpretation; packet-byte overhead not captured
Case 1	Topic normalized-goodput table and provided throughput graph	Measured redundant bytes and overhead graph
Case 2	Automatic-recovery goodput table and graph	Retry-count graph; byte retransmission not recoverable
Case 2 Enhanced	Parallel goodput table and graph	Retry-count graph; byte retransmission not recoverable

10 Conclusion

The supplied project demonstrates the progression from functional TCP topic distribution to server concurrency, probabilistic failures, restart-based recovery, persisted checkpoints and parallel resume. Successful transfers and archived measurements substantiate the core workflow. The recorded results illustrate both the overhead of repeated transmission and the potential of concurrency, while remaining sensitive to load, network conditions and fault parameters. The report therefore preserves the observed data without filling unmeasured conditions or treating dissimilar trials as directly equivalent. Before final submission, the team should integrate the separate code handoffs, repeat matched benchmarks and attach complete file-integrity evidence.